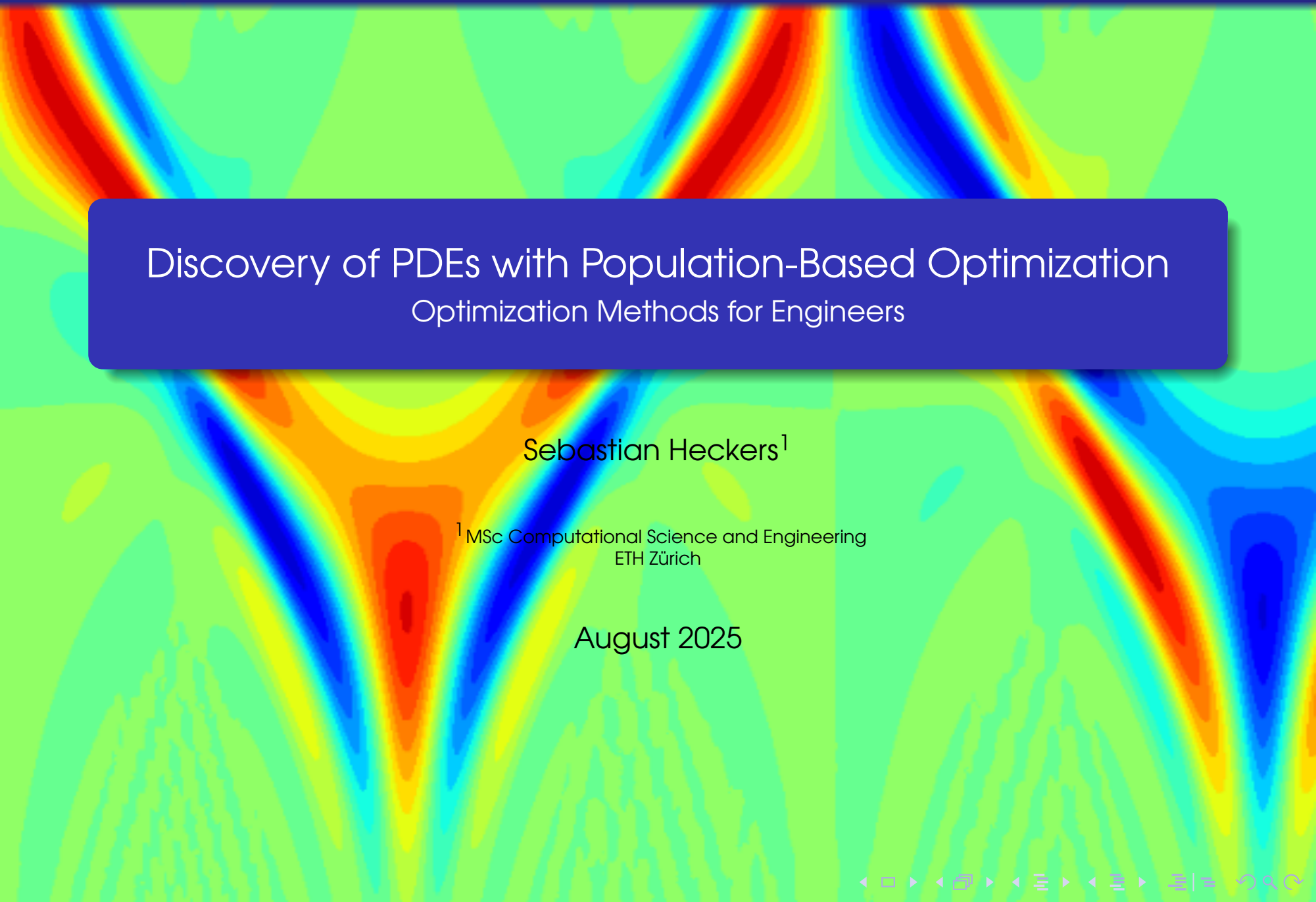


Discovery of PDEs with Population-Based Optimization

Optimization Methods for Engineers

Sebastian Heckers[1]

[1] MSc Computational Science and Engineering
ETH Zürich

August 2025

Motivation

Partial Differential Equations (PDE) are an important part of science. Therefore, it gives many methods to derive PDEs, as well as data-driven methods. These methods use spatio-temporal data and regression frameworks to find the fitting equation.

A specific method, PDEFind, for PDEs was developed in (1) using sparse regression. But this method is computationally expensive and is therefore limited to a smaller function space. To increase the capabilities population-based optimization methods are introduced.

APS/123-QED

Data-driven discovery of partial differential equations

Samuel H. Rudy[1]*, Steven L. Brunton[2], Joshua L. Proctor[3], and J. Nathan Kutz[1]
[1] *Department of Applied Mathematics, University of Washington, Seattle, WA. 98195*
[2] *Department of Mechanical Engineering, University of Washington, Seattle, WA. 98195 and*
[3] *Institute for Disease Modeling, , 3150 139th Ave SE, Bellevue, WA 98005*
(Dated: September 22, 2016)

We propose a sparse regression method capable of discovering the governing partial differential equation(s) of a given system by time series measurements in the spatial domain. The regression framework relies on sparsity promoting techniques to select the nonlinear and partial derivative terms terms of the governing equations that most accurately represent the data, bypassing a combinatorially large search through all possible candidate models. The method balances model complexity and regression accuracy by selecting a parsimonious model via Pareto analysis. Time series measurements can be made in an Eulerian framework where the sensors are fixed spatially, or in a Lagrangian framework where the sensors move with the dynamics. The method is computationally efficient, robust, and demonstrated to work on a variety of canonical problems of mathematical physics including Navier-Stokes, the quantum harmonic oscillator, and the diffusion equation. Moreover, the method is capable of disambiguating between potentially non-unique dynamical terms by using multiple time series taken with different initial data. Thus for a traveling wave, the method can distinguish between a linear wave equation or the Korteweg-deVries equation, for instance. The method provides a promising new technique for discovering governing equations and physical laws in parametrized spatio-temporal systems where first-principles derivations are intractable.

PACS numbers: 05.45.-a, 05.45.Yv

Data-driven discovery methods, which have been enabled in the last decade by the plummeting cost of sensors, data storage and computational resources, are having a transformative impact on the sciences, enabling a variety of innovations for characterizing high dimensional data generated from experiments. Less well understood is how to uncover underlying physical laws and/or governing equations from time series data that exhibit spatio-temporal activity. Traditional theoretical methods for deriving the underlying partial differential equations (PDEs) are rooted in conservation laws, physical principles and/or phenomenological behaviors. These first principle derivations lead to many of the canonical models of mathematical physics. However, there remain many complex systems that have eluded quantitative analytic descriptions or even characterization of a suitable choice of variables (e.g. neuroscience, power grid, epidemiology, finance, ecology, etc). We propose an alternative method to derive governing equations based solely on time series data collected at a fixed number of spatial locations. Using innovations in sparse regression, we discover the terms of the governing PDE that most accurately represent the data from a large library of potential candidate functions. Importantly, measurements can be made in an Eulerian framework where the sensors are fixed spatially, or in a Lagrangian framework where the sensors move with the dynamics. We demonstrate the success of the method by deriving, from time series data alone, many canonical models of mathematical physics.

Methods for data-driven discovery of dynamical systems [1] include equation-free modeling [2], empirical dynamic modeling [3, 4], modeling emergent behavior [5], and automated inference of dynamics [6–8]. In this series of developments, seminal contributions leveraging symbolic regression and an evolutionary algorithm [9, 10] were capable of directly determining nonlinear dynamical system from data. More recently, sparsity promoting techniques [13] have been used to robustly determine, in a highly efficient computational manner, the governing dynamical system [12, 13]. Both the evolutionary [10] and sparse [12] symbolic regression methods avoid overfitting by selecting parsimonious models that balance model accuracy with complexity via Pareto analysis.

The method we present is able to select linear and/or nonlinear terms, including spatial derivatives, resulting in the identification of PDEs from data. Previous methods are able to identify ODEs from data, but not partial derivative terms [12]. Only those terms that are most informative about the dynamics are selected as part of the discovered PDE. The generalization presented here is critically important since the majority of canonical models in mathematical physics contain spatio-temporal dynamics. The addition of spatial structure is nontrivial and requires modification of the methodology and data collection (Eulerian or Lagrangian measurements) in order to circumvent potential ambiguities. The resulting algorithm, *PDE functional identification of nonlinear dynamics* (PDE-FIND), is applied to numerous canonical models of mathematical physics.

In what follows, we consider a PDE of the form

$$u_t = N(u, u_x, u_{xx}, \cdots, x, \mu) \quad (1)$$

where the subscripts denote partial differentiation in either time or space, and $N(\cdot)$ is an unknown right-hand

*Electronic address: shrudy@uw.edu

Front page from (1).

Problem Description

Given are spatial, time series data with N data points in $\mathbb{R}^2$.
Find a term for the underlying PDE which minimizes the approximation error.

The given data and additional inputs are fit into column vectors $U, Q \in \mathbb{R}^N$. A library $\Theta(U, Q) \in \mathbb{R}^{N \times D}$ is constructed out of D candidates of linear, nonlinear terms and partial derivatives. This library is for the PDE:

$$U_t = \Theta(U, Q)\xi$$

With the time derivative U_t and the coefficients $\xi \in \mathbb{R}^D$ for each candidate. Find ξ which minimizes

$$\xi^* = \text{argmin}_\xi ||\Theta(U, Q)\xi - U_t||_2$$

The non-linear cost function $f(\xi; \Theta, U_t) = ||\Theta\xi - U_t||_2 + \kappa||\xi||_0$ has a data term and an L1 regularization term, which penalizes many nonzero coefficients.

Problem Objectives

Use new optimization methods instead of the regression framework for the coefficients. Namely, the population-based optimization methods **Genetic Algorithm** (GA) and **Particle Swarm Optimization** (PSO) based on (2). These are the main goal:

- Implement classical GA with deap(3).
- Implement PSO with specific forces and modifications.
- Investigate the parameters for both algorithms.
- Compare the performance of the population-based methods to PDEFind.

Monte-Carlo (MC)

The Monte-Carlo method samples random coefficient vectors ξ and is used as baseline method.

1. Initialization: create M uniformly distributed coefficient vectors.
2. Evaluation: calculate the fitness for each vector and find the best.
3. Result: return the vector with the best result ξ^*.

For the comparison with other methods, the total number of vectors M is the product of the number of iterations and number of particles or individuals, $M = N \cdot n_{iters}$, to have a similar number of fitness evaluations.

Particle Swarm Optimization (PSO)

N particles move in the continuous search space $\mathbb{R}^D$, evaluate the fitness $f(\xi)$ and go towards an optimal position. Each particle represents a possible solution ξ and has a position p_m and velocity v_m for $m = 1, ..., N$.

1. Initialization: sample N uniformly distributed particles $p_m^0, v_m^0 \in \mathbb{R}^D$ with zero velocity.
2. Iteration: repeat the following steps.
   1. Evaluation: calculate the fitness of each particle, $f(p_m^t)$.
   2. Analysis: interpret and evaluate the current state of all particles, or stop with a sufficient solution.
   3. Update: calculate forces and update the position p_m^{t+1} and velocity v_m^{t+1} of each particle.

$$v_m^{t+1} = w \cdot v_m^t + a_{ind} R_N \cdot (p_m^* - p_m) + a_{glb} R_N \cdot (p^* - p_m) + a_{rnd} \cdot v_{rnd} + a_{com} R_N \cdot v_{com}$$

$$p_m^{t+1} = p_m^t + v_m^{t+1}$$

3. Result: return the particle with the best result p^*.

Forces

The forces are the terms in the update of the velocities v_m^t.

$$v_m^{t+1} = w \cdot v_m^t + a_{ind} R_N \cdot (p_m^* - p_m) + a_{glb} R_N \cdot (p^* - p_m) + a_{rnd} \cdot v_{rnd} + a_{com} R_N \cdot v_{com}$$

- $w \cdot v_m^t$: the inertia of the particle
- $a_{ind} R_N \cdot (p_m^* - p_m)$: the attraction to the individual best position p_m^*
- $a_{glb} R_N \cdot (p^* - p_m)$: the attraction to the global best position p^*
- $a_{rnd} \cdot v_{rnd}$: the random perturbations for each coefficient, $v_{rnd} \in [0, 1]^{N \times D}$
- $a_{com} R_N \cdot v_{com}$: the repelling force from the center of the particles

w, a_{ind}, a_{glb}, a_{rnd} and a_{com} are the weights for each force, and $R_N \in [0, 1]^N$ is the random component.

Modifications

The **convergence factor** c is a scaling factor that reduces the variations of forces. After an initial period with $c = 1$ the factor scales down depending on the number of iterations without improvements, $c \sim 1/n_{\text{no improvement}}$. This reduces perturbations from repelling forces to converge towards an optimal position.

The repelling force from the centre of particles v_{com} is a simplified version of the repelling forces to its neighbours. To avoid large exchanges between particles, only the centre of mass and the variations of particles are used. This force is scaled by the inverse of the variations of the particles to avoid the particles accumulating in a local optimum.

Genetic Algorithm (GA)

The GA has a population with N individuals with an integer string of length D representing a possible solution ξ. It uses crossover and mutation to produce new children.

1. Initialization: create and evaluate population P with uniformly distributed individuals.
2. Iteration: repeat following steps until maximum iterations.
   1. Selection: select individuals with low fitness to create new children.
   2. Crossover: combine the genome strings of 2 individuals with probability a_{cxpb}% (two-point-crossover).
   3. Mutation: mutate genes of individuals with probability a_{mutpb}% (flip-bit).
   4. Evaluation: calculate and assemble the fitness of new individuals.
3. Result: return the individual with the best result $p*$.

Implemented with deap (3).

Modifications

The *cost function* returns the inverse, $f'(p_m) = 1/f(p_m)$, and the deap framework searches the maximum fitness.

For the *selection* the deap tool `selTournament()` is used. It selects the best individuals out of random groups of the population.

The *population size* is constant. The population contains the same number of individuals in each iteration step.

Parametric Optimization

The PSO and GA algorithm have different variable parameters:

N	number of particles
n_{iters}	number of iterations
D	dimension of coefficients
w	weight of inertia
a_{ind}	weight of individual best
a_{glb}	weight of global best
a_{rnd}	weight of random perturbations
a_{com}	weight of repelling force

Table: Variable parameters of PSO.

N	number of individuals
n_{iters}	number of iterations
D	dimension of coefficients
a_{cxpb}	probability of crossover
a_{mutpb}	probability of mutations

Table: Variable parameters of GA.

The parameters N, n_{iters} and D are fixed. They are chosen to be sufficiently large and not too computationally expensive. The other parameters are tested inside a reasonable range with several repetitions.

PSO Parameters

The plots show the average best results. They display the value of the cost function, data approximation and the distance to the correct PDE formulation.

The fixed parameters are $N = 200$, $N_{iters} = 1000$ and $D = 21$. The results for the parametric optimization of PSO are noisy, but follwing values were choosen: $w = 0.1$, $a_{ind} = 2$, $a_{glb} = 1.5$, $a_{rnd} = 0.4$ and $a_{com} = 0.5$.

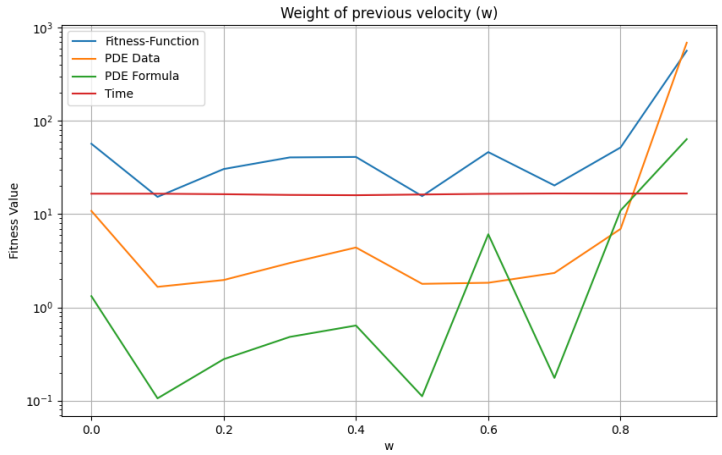
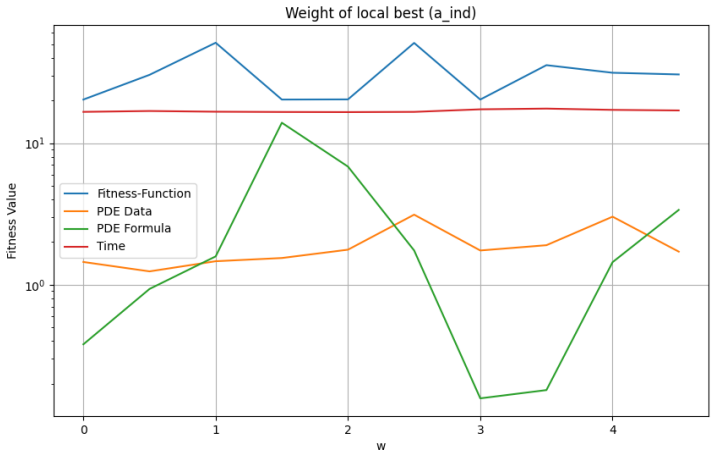


Figure: Plot for PSO parameteric optimization of w.



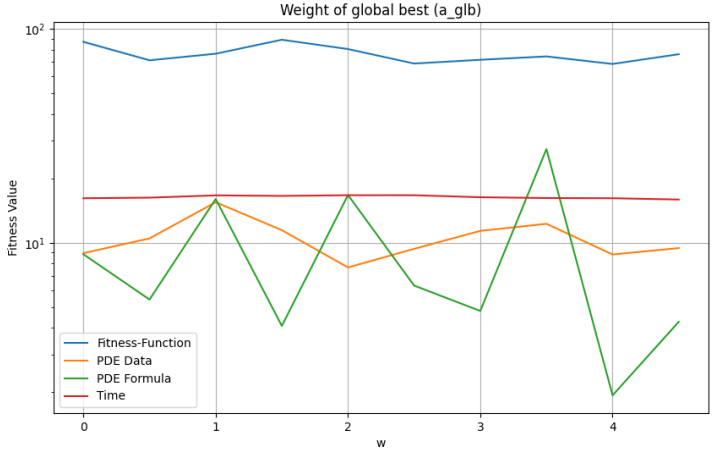
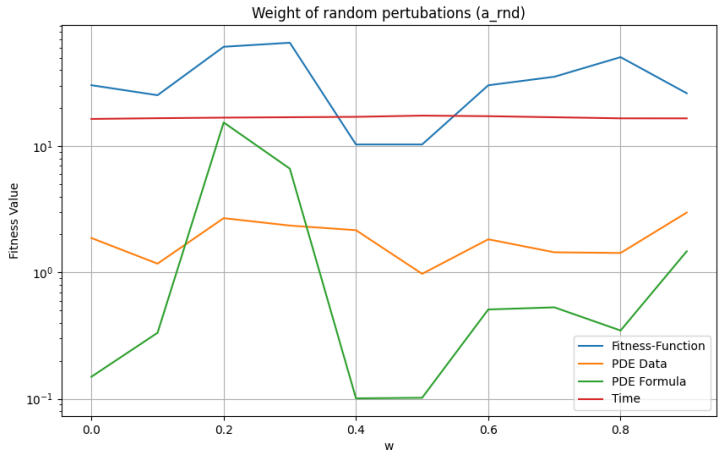


Figure: Plots for PSO parameters a_{ind} and a_{glb}.



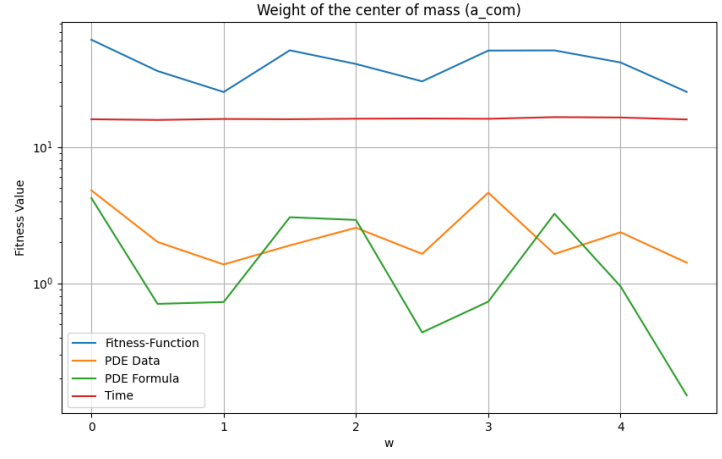


Figure: Plots for PSO parameters a_{rnd} and a_{com}.

GA Parameters

The plots show the average best results. They display the value of the cost function, data approximation and the distance to the correct PDE formulation.

The fixed parameters are $N = 200$, $N_{iters} = 500$ and $D = 21$. The results for the parametric optimization of PSO are noisy, but follwing values were choosen: $a_{cxpb} = 0.5$ and $a_{mutpb} = 0.5$.

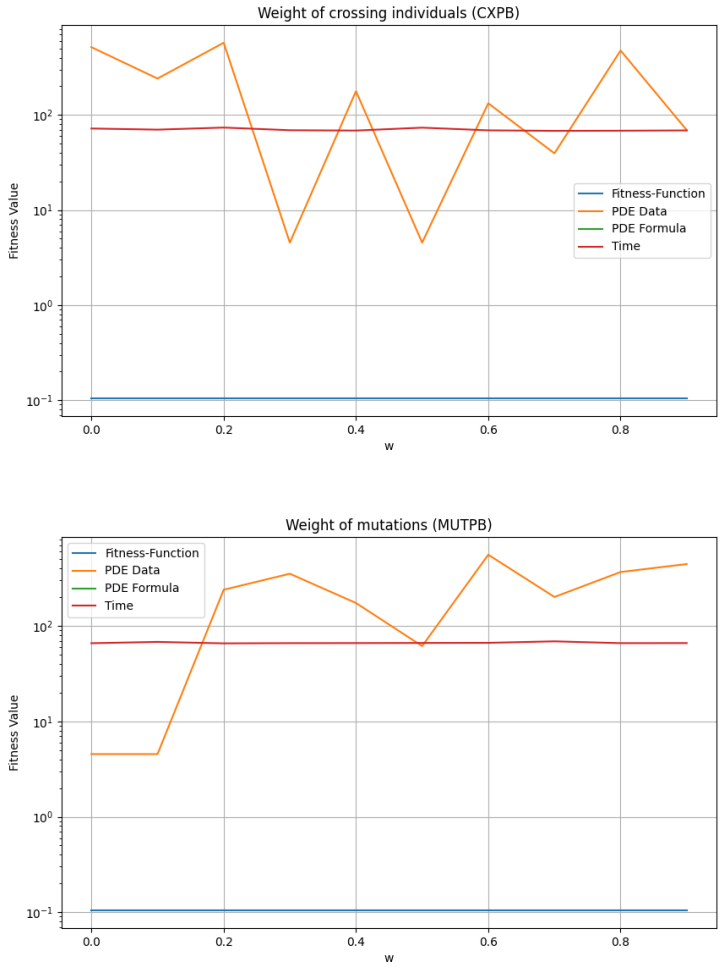


Figure: Plots for GA parametric optimization.

Results

The different algorithms are tested on 3 given datasets[a]. The goal is to find the right candidate terms for the underlying PDEs.

The algorithms PSO, GA and PDEFind are able to find a good approximation for data points (see figure), but often fail to find the exact candidate terms for the PDE formulation.

Each algorithm was tested and finetuned on each dataset. Below are the general results with similar settings for each dataset.

[a] dataset from (1) on https://github.com/snagcliffs/PDE-FIND

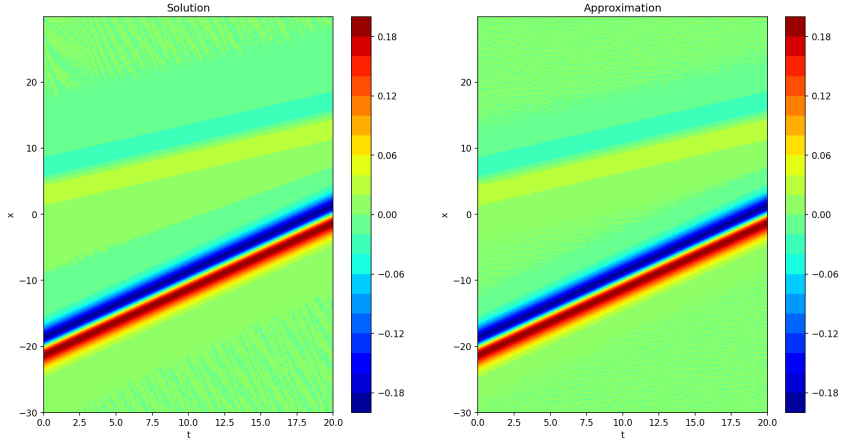


Figure: Visualization of an approximation of the data by the PDEFind method.

Dataset 1

This dataset is from the **Burgers equation**

$$u_t = -0.1u_{xx} + u \cdot u_x$$

with 256 grid points and 101 time steps.

The algorithm can approximate the data relative good and come close to the right formulation.

PDEFind	PSO	GA	MC
10.310	10.860	122.401	4'026.267
0.03	0.4	0.4	370

Table: Mean fitness and best relative error for each method.

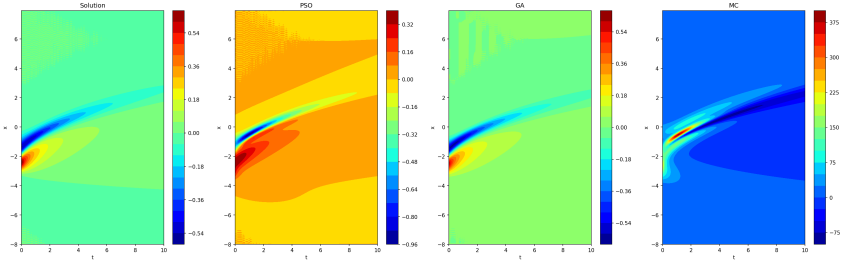


Figure: Visualization of dataset 1 and the best results of PSO, GA and MC.

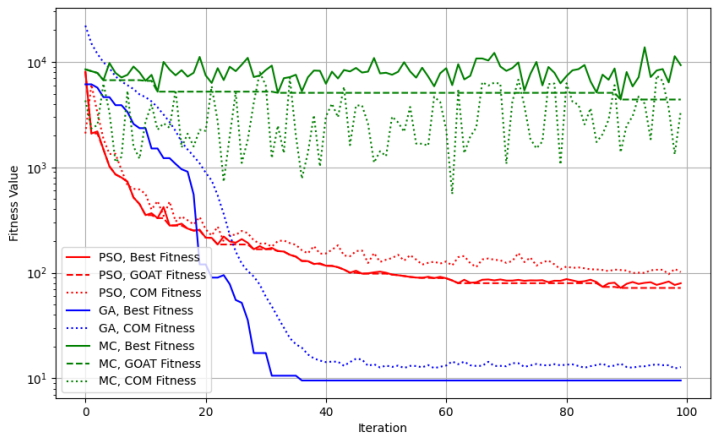


Figure: Plot of fitness value during iterations for dataset 1.

Dataset 2

This dataset is from the **Korteweg-de Vries equation**

$$u_t = u_{xxx} + 6 \cdot u \cdot u_x$$

with 512 grid points and 201 time steps.

Approximating the data and formulation is a bit harder for the algorithms.

PDEFind	PSO	GA	MC
40.689	11.789	30.789	569.219
0.05	0.3	0.2	31

Table: Mean fitness and best relative error for each method.

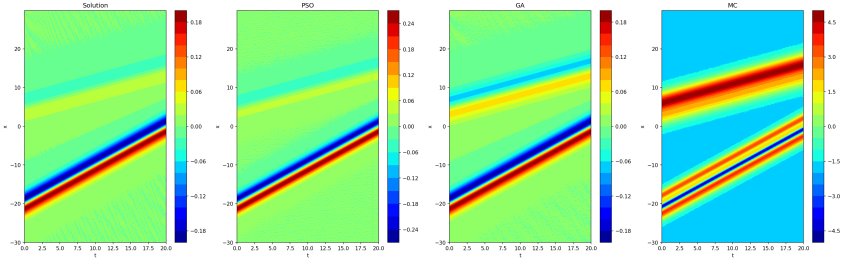


Figure: Visualization of dataset 2 and the best results of PSO, GA and MC.

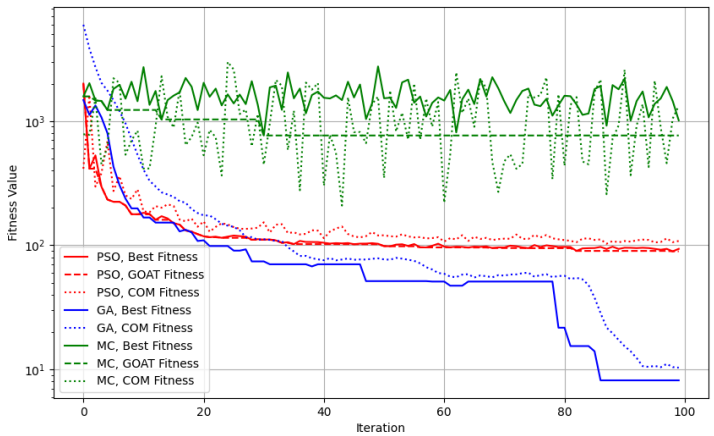


Figure: Plot of fitness value during iterations for dataset 2.

Dataset 3

This dataset is from the **Schrödingers equation**

$$iu_t = -0.5 \cdot u_{xx} + \frac{x}{2}u$$

with 512 grid points and 401 time steps.

This dataset is the most difficult dataset to approximate by all algorithms.

PDEFind	PSO	GA	MC
6'761.942	336.769	354.946	62'051.71
20	0.9	1	150

Table: Mean fitness and best relative error for each method.

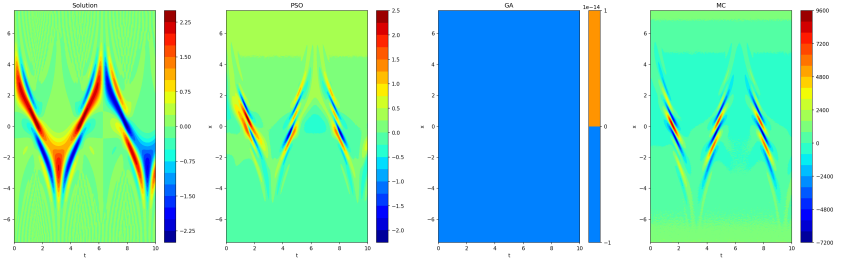


Figure: Visualization of dataset 3 and the best results of PSO, GA and MC.

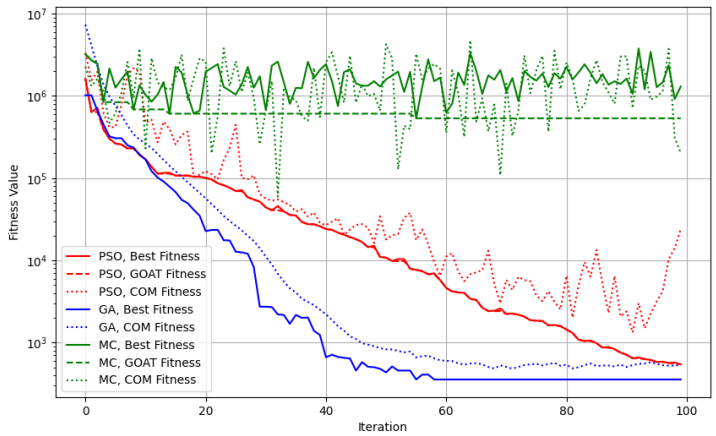


Figure: Plot of fitness value during iterations for dataset 3.

Runtime

The runtime strongly depends on the number of data points and candidate terms. The PSO is clearly faster then GA and similar to MC. PDEFind is significantly faster then the other methods with 10 iterations.

	Dataset 1	Dataset 2	Dataset 3
# data points	25856	102912	205312
PSO	21.279	65.989	261.204
GA	84.590	341.526	362.192
MC	24.868	65.634	256.125
PDEFind	0.030	0.090	0.890

Table: Runtime in seconds for each method and dataset with $D = 21$.

PDEFind used a least-square technique to find the coefficients more directly and reduces the search space by eliminating some candidates. This makes the iteration step highly efficient. Meanwhile the classic population-based methods need a large number of iterations and cost evaluations, that makes it more computational expensive.

Discussion

Even though the fitness landscape is very large and complex with many local optima the **Monte-Carlo** method does not come close to the solution and the error is usually over 1-2 magnitudes larger.

The **Particle-Swarm Optimization** decreases the error very quickly at the beginning and then decreases slowly in the iterations afterwards. This allows it to find a good approximation to the data points with sufficient iterations and particles, but it tends to "overfit" the coefficients. The resulting PDE term often contains to many terms, which are not necessary and even cancel each other out. This is a typical situation of getting stuck in a local minimum.

The **Genetic Algorithm** doesn't improves generally as fast as PSO does, but always have significant jumps of improvement. The GA better reduces the number of candidates and finds other optimums, but it tends to "underfit" the coefficients. Meaning it almost always removes to many candidates resulting in a naive solution with few to no terms. This can be solved by finetuning the cost function with the lasso regularization.

As seen in the results and runtime the **PDEFind** method often outperforms the other methods. Its highly efficient iteration step has a short runtime, but it eliminates candidates from the search space and does so often excludes the correct solution.

Testing these methods showed that the best way to use PSO, GA and PDEFFind is to finetune the parameters of each algorithm, the candidate terms and lasso regularization. The candidate terms used determine the search space. Reducing them improves the result and decreases the runtime, but it limits the range and correctness. Optimally a large search space with all possible candidates would be used, but this is too computational complex. An educated guess is the best way to narrow the search space and improve the performance.

Future work to improve these methods could implement an hybrid method of a population-based method and a least-square technique. This approach combines the short runtime and exacter coefficients determination with the additional information about the fitness landscape and random perturbations. Or use Symbolic regression, this is the current state-of-art to find analytic expression, see (4). With its tree-based representation it handles the challenges, like many local optima, better than ridge regression or other methods.

Conclusion

The results demonstrate that while methods like PDEFind are highly efficient in runtime, their constrained search space can hinder discovery of complex formulations. On the other hand, population-based methods, though computationally more demanding, provide a more flexible and global exploration of the solution space. PSO, in particular, showed strong performance in data approximation, albeit with a tendency toward overfitting. GA, by contrast, excelled in structural sparsity but suffered from underfitting. Overall, this project highlights the trade-offs between efficiency, accuracy, and generalizability in PDE discovery. The integration of domain knowledge and careful parameter tuning remains essential.

References

S. H. Rudy, S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Data-driven discovery of partial differential equations," 2016.

P. Leuchtmann, "Optimization for engineers," 2020.

F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, jul 2012.

F. O. de Franca, M. Virgolin, M. Kommenda, M. S. Majumder, M. Cranmer, G. Espada, L. Ingelse, A. Fonseca, M. Landajuela, B. Petersen, R. Glatt, N. Mundhenk, C. S. Lee, J. D. Hochhalter, D. L. Randall, P. Kamienny, H. Zhang, G. Dick, A. Simon, B. Burlacu, J. Kasak, M. Machado, C. Wilstrup, and W. G. L. Cava, "Interpretable symbolic regression for data science: Analysis of the 2022 competition," 2023.